

IMPLEMENTATION- OF DEVELOPMENT PLAN

AI-Agent (Auto part Storage)



Groepsnaam: Fatneek

Docent: Dhr. Christian MS CS/ Mevr. Bergwijn

Sem: 5.2

Groepsleden:

- Ramkhelawan Riaaz
- Sodipo Sherreskly
- Sangham Rishika
- Ramdhiansing Shakeel
- Sariman Jalen

Inhoudsopgave

1. Deploymentstappen (cloud of lokaal)	3
2. Omgevingen (dev/test/prod) (bonus)	4
2.1 Development omgeving	4
2.2 Test omgeving	4
2.3 Productie omgeving.....	5
3. Configuratiebeheer	6
4. Versiebeheerstrategie	6
5. Updateprocedure en rollbackstrategie	7
5.1 Updateprocedure	7
5.2 Monitoring en logging	7
5.3 Rollbackstrategie	7
6. Beheer en verantwoordelijkheden	8
6.1 Verantwoordelijkheden.....	8
6.2 Onderhoud en support.....	9
7. Requirements en kosteninschatting	10
7.1 Requirements	10
7.2 Indicatieve kosten bij klein gebruik.....	11

Lijst met Tabellen

Tabel 1: Kosteninschatting.....	11
---------------------------------	----

1. Deploymentstappen (cloud of lokaal)

De applicatie is gebouwd met Next.js 16, waarbij frontend en backend binnen 1 geïntegreerd framework worden beheerd. Het draait volledig in een cloudomgeving en wordt gehost via Vercel en gebruikt een serverless PostgreSQL-database van Neon (<https://neon.com/>).

Applicatie setup en deployment:

1. Lokaal systeem voorbereiden

- Zorg dat Node.js is geïnstalleerd (Next.js 16 draait op Node.js 18+)
- Zorg dat npm geïnstalleerd is
- Zorg dat je toegang hebt tot de Neon Database en dat credentials beschikbaar zijn

2. Code ophalen van GitHub

- Open VS-code
- Clone de repository: <https://github.com/Shwek07/autobot.git>

3. Dependencies installeren

- Installeer de benodigde packages: `npm install`

4. Configuratiebestanden plaatsen

- Maak een `.env` bestand en vul het met Neon Database gegevens:
`DATABASE_URL= *de gegevens*`

5. Database configureren (Neon)

- Log in bij Neon met een e-mail en controleer de database.

6. Applicatie lokaal draaien (testen)

- Open de terminal in VS Code
- Start de development server met het commando: `npm run dev`
- Open de browser en ga naar <http://localhost:3000> om de app te testen
- Controleer of de applicatie correct verbinding maakt met de database van Neon
- Er is gebruik gemaakt van NextAuth in combinatie met Google Authentication, je kan gewoon met je bestaande Google account inloggen.

7. Deployen naar Vercel

- Installeer Vercel CLI: `npm i -g vercel` → `vercel login`.
- Deploy de applicatie met: `vercel` (hiermee wordt de code naar Vercel gestuurd en een nieuwe build start)
- Voeg dezelfde environment variables toe in het Vercel dashboard (zoals in `.env`).
- Vercel genereert automatisch een SSL-certificaat, zodat je website veilig via HTTPS bereikbaar is.

2. Omgevingen (dev/test/prod) (bonus)

2.1 Development omgeving

De development omgeving wordt gebruikt door de ontwikkelaars om de applicatie lokaal te ontwikkelen en te testen. De broncode wordt beheerd via GitHub en lokaal geopend in Visual Studio Code. Ontwikkelaars kunnen hiermee wijzigingen aanbrengen en nieuwe functionaliteiten implementeren voordat deze naar de centrale repository worden gepusht.

Werkwijze bij development:

1. De repository wordt gekloond via de GitHub URL
2. Het project wordt geopend in Visual Studio Code.
3. De benodigde dependencies worden geïnstalleerd.
4. De applicatie wordt lokaal gestart voor ontwikkeling en debugging

2.2 Test omgeving

De applicatie wordt getest met behulp van de previewomgeving van Vercel. Wanneer nieuwe code naar de GitHub-repository wordt gepusht, start Vercel automatisch een buildproces en wordt er een preview deployment gegenereerd met een unieke URL. Deze deployment bevat de nieuwste versie van de applicatie en wordt gebruikt om wijzigingen te testen voordat ze naar de productieomgeving worden uitgerold.

Werkwijze bij testproces:

1. Ontwikkelaars halen de laatste code op vanuit GitHub
2. De applicatie wordt lokaal gestart
3. Wijzigingen worden naar repository gepusht
4. Vercel genereert automatisch een previewversie van de applicatie
5. Er wordt gecontroleerd of alle functies correct werken en of de applicatie verbinding kan maken met de database van Neon
6. Eventuele bugs of configuratieproblemen worden opgelost.

Hiermee wordt getest of de applicatie goed werkt op verschillende systemen en configuraties.

Strategie voor een efficiënte testomgeving

Voor komende projecten zal een betere testomgeving worden ingericht. Dit omvat:

1. Het integreren van geautomatiseerde tests (unit-, integratie- en end-to-end tests)
2. Het gebruik van gestandaardiseerde testdata en configuraties.

2.3 Productie omgeving

De productieomgeving is de live omgeving waarin de applicatie beschikbaar is voor eindgebruikers. Deze omgeving draait op Vercel, dat zorgt voor hosting, builds en automatische deployment van de meest stabiele versie van de applicatie.

Werkwijze in productie

1. Ontwikkelaars testen en ontwikkelen nieuwe functies lokaal (development)
2. Code wordt via GitHub naar main branch gepusht
3. Vercel detecteert de push en start automatisch een build
4. Na een succesvolle build wordt de nieuwe versie live gezet voor eindgebruikers
5. Monitoring en logging zorgen ervoor dat eventuele problemen snel worden opgespoord en opgelost

Belangrijke nota:

- Alleen code die succesvol is getest in de development omgeving en gemerged naar de main branch, wordt uitgerold naar productie.
- Vercel houdt build- en deploymentlogs bij voor foutdetectie

Schaalbaarheid

Voor een klein aantal gebruikers volstaan de gratis versies van Vercel, Neon Database en GROQ. Zodra de applicatie echter door meerdere gebruikers tegelijk wordt gebruikt, er meer data verwerkt moet worden of complexere GROQ queries nodig zijn, zal de worden overgestapt naar de betaalde versies van Vercel, Neon en/of GROQ. Dit biedt:

- Extra server- of databasecapaciteit
- Hogere query- en opslaglimieten
- Betere prestaties en betrouwbaarheid
- Mogelijkheid om meer gelijktijdige gebruikers te bedienen

Door deze opschaling blijft de productieomgeving stabiel, veilig en continu beschikbaar voor eindgebruikers.

3. Configuratiebeheer

Configuratiebeheer gaat over het veilig beheren van instellingen en gevoelige gegevens die nodig zijn voor de applicatie. Dit is in de applicatie als volgt geregeld:

1. Environment variables:

Gevoelige gegevens, zoals databaseverbindingen, API sleutels en authenticatiesleutels, worden niet in de codebase opgeslagen maar beheerd via environment variables.

- Lokaal worden deze opgeslagen in een .env-bestand.
- In productie worden deze ingesteld via de beveiligde environment settings van Vercel.

2. Beveiliging:

Wachtwoorden, sleutels en overige gevoelige configuratiegegevens zijn niet publiekelijk toegankelijk en maken geen onderdeel uit van de repository.

3. Monitoring

Serverlogs worden gemonitord via Vercel. Database-activiteiten en eventuele foutmeldingen worden gecontroleerd via het Neon-dashboard. Hierdoor kunnen configuratie- en runtimeproblemen tijdig worden gedetecteerd.

4. Versiebeheerstrategie

Versiebeheer betreft het bijhouden van codewijzigingen, het veilig ontwikkelen van nieuwe functionaliteiten en het beheren van stabiele versies van de applicatie.

Vercel biedt een geïntegreerd dashboard waarmee developers eenvoudig het versiebeheer van hun webapplicaties kunnen beheren en monitoren. Elke wijziging in de code die naar GitHub wordt gepusht, genereert automatisch een *Preview Deployment*. Dit betekent dat elke feature branch of pull request een aparte, tijdelijke versie van de applicatie krijgt, die beschikbaar is via een unieke URL.

Het dashboard toont developers een overzicht van alle deploys, inclusief:

- **Versienummer of commit ID:** identificeert de specifieke build
- **Branch:** de bronbranch van de code (bijv. main, feature)
- **Status:** geslaagd, in progress of mislukt
- **Tijdstempel:** wanneer de deploy is uitgevoerd

Door middel van dit overzicht kunnen developers:

1. Nieuwe versies en eerdere versies bekijken via de preview links of het dashboard, zonder dat de live productieomgeving beïnvloed wordt.
2. Testen en valideren van features voordat ze in productie gaan.

3. Rollbacken naar een vorige versie indien een nieuwe release problemen veroorzaakt.

5. Updateprocedure en rollbackstrategie

5.1 Updateprocedure

Nieuwe versies van de applicatie worden automatisch uitgerold via Vercel. Zodra een feature-branch wordt gemerged naar de main-branch, start Vercel automatisch het buildproces. Tijdens deze build worden alle afhankelijkheden geïnstalleerd en wordt de applicatie klaargemaakt voor online publicatie.

Na een succesvolle build wordt de nieuwe versie automatisch gepubliceerd, zodat gebruikers direct toegang hebben tot de nieuwste functionaliteit zonder handmatige interventie. Dit proces zorgt voor efficiënte en betrouwbare updates, terwijl de productieomgeving continue beschikbaar blijft.

5.2 Monitoring en logging

Vercel registreert alle build- en deploymentlogs, waardoor het team inzicht krijgt in de status van builds en eventuele fouten kan analyseren.

Het Neon Dashboard monitort daarnaast het databasegebruik en de queryactiviteit, zodat database-gerelateerde problemen snel opgespoord kunnen worden

Dankzij deze monitoring kunnen developers bij onverwachte problemen snel reageren en indien nodig een eerdere stabiele versie activeren, zodat updates veilig en gecontroleerd plaatsvinden.

5.3 Rollbackstrategie

Bij problemen in de productieomgeving kan via het Vercel-dashboard direct worden teruggekeerd naar een stabiele eerdere deployment. Indien nodig kan ook een specifieke commit via Git worden hersteld en opnieuw gedeployed.

Dit rollbackmechanisme beperkt de impact op gebruikers en zorgt ervoor dat de applicatie binnen korte tijd weer volledig functioneel is.

6. Beheer en verantwoordelijkheden

6.1 Verantwoordelijkheden

Verantwoordelijkheden zijn de belangrijkste taken en bevoegdheden die een teamlid heeft binnen het project. Deze taken zijn vooral gericht op het ontwerpen en ontwikkelen van het systeem. Hierbij worden beslissingen genomen over technologie, structuur en functionaliteit van de applicatie. Het doel is om een stabiel, schaalbaar en goed functionerend systeem te bouwen dat voldoet aan de eisen van het project.

- **Software Architect – Riaaz**

De software architect is verantwoordelijk voor het algemene ontwerp van het systeem. Deze rol bepaalt de technische architectuur en maakt keuzes over welke technologieën worden gebruikt, zoals Vercel, Neon en GROQ. Daarnaast bewaakt de software architect de structuur van het systeem en zorgt hij ervoor dat alle onderdelen efficiënt samenwerken.

- **Backend & API developer – Jalen**

De backend developer ontwikkelt en beheert de serverlogica en API's van de applicatie. Deze rol zorgt ervoor dat de frontend correct kan communiceren met de database en dat gegevens veilig en efficiënt worden verwerkt.

- **Database Engineer – Rishika**

De database engineer ontwerpt en beheert de structuur van de database. Dit omvat het opzetten van datamodellen, het organiseren van gegevens en het zorgen dat data snel en efficiënt kan worden opgeslagen en opgehaald.

- **Frontend & UI/UX Designer – Shakeel**

De frontend en UI/UX designer ontwerpt en ontwikkelt de gebruikersinterface van de applicatie. Het doel is om een interface te creëren die gebruiksvriendelijk, duidelijk en visueel aantrekkelijk is voor de eindgebruikers.

- **Technische Schrijver – Sherreskly**

De technische schrijver is verantwoordelijk voor het documenteren van het project. Dit omvat handleidingen, technische documentatie en deployment guides. Deze documentatie helpt ontwikkelaars en gebruikers om het systeem beter te begrijpen en te gebruiken.

6.2 Onderhoud en support

Onderhoud & support omvat de taken die nodig zijn om het systeem stabiel, veilig en goed werkend te houden nadat het is ontwikkeld. Dit betekent dat fouten worden opgelost, updates worden uitgevoerd en prestaties van het systeem worden gecontroleerd. Het doel van onderhoud en support is om ervoor te zorgen dat gebruikers de applicatie zonder problemen kunnen blijven gebruiken.

- **Software Architect – Riaaz**

De software architect controleert of nieuwe updates of wijzigingen geen negatieve invloed hebben op de prestaties, schaalbaarheid of stabiliteit van het systeem. Indien nodig worden verbeteringen voorgesteld om de architectuur te optimaliseren.

- **Backend & API Developer – Jalen**

De backend developer lost fouten op in de serverlogica en API's. Daarnaast voert hij security updates uit en controleert serverlogs om mogelijke problemen vroegtijdig te detecteren.

- **Database Engineer – Rishika**

De database engineer zorgt voor regelmatig onderhoud van de database. Dit omvat het maken van back-ups, het monitoren van databaseprestaties en het optimaliseren van queries om ervoor te zorgen dat data snel en veilig kan worden verwerkt.

- **Frontend & UI/UX Designer – Shakeel**

De frontend designer lost problemen in de gebruikersinterface op, zoals bugs of compatibiliteitsproblemen met verschillende apparaten of browsers. Daarnaast worden verbeteringen aangebracht op basis van feedback van gebruikers.

- **Technische Schrijver - Sherreskly**

De technische schrijver houdt alle documentatie van het project up-to-date wanneer er wijzigingen of nieuwe functionaliteiten worden toegevoegd. Daarnaast worden klachten, vragen en feedback van gebruikers bij deze rol gemeld en geregistreerd. Deze feedback wordt vervolgens doorgegeven aan de juiste ontwikkelaars, zodat problemen kunnen worden geanalyseerd en opgelost. Hierdoor verloopt de communicatie tussen gebruikers en het ontwikkelteam gestructureerd.

7. Requirements en kosteninschatting

7.1 Requirements

De SDS-applicatie draait volledig via Vercel, een cloud-based platform dat zowel de frontend hosting als de serverless backend functies verzorgt. Hierdoor is het gebruik van een eigen server, CPU of extra RAM door de gebruiker niet nodig.

Vercel beheert de benodigde compute resources in de cloud:

- Serverless functies verwerken requests en communiceren met de database en eventuele externe APIs.
- Schaalbare infrastructuur zorgt dat de applicatie automatisch kan opschalen bij hogere belasting, zonder dat de gebruiker iets hoeft te installeren of aan te passen.
- CDN en performance-optimalisatie garanderen snelle laadtijden, ongeacht de hardware van het apparaat van de gebruiker.

Niet alle componenten worden door Vercel geleverd. Voor de volledige werking van de SDS-applicatie zijn aanvullende resources nodig:

- **Database:** Neon Database wordt gebruikt voor het opslaan van gebruikersdata en logs. Vercel verzorgt geen opslag of persistente databases.
- **Runtime-omgevingen:** Node.js is nodig voor lokale development en testen van backend-logica.
- **Ontwikkelomgeving:** VS Code wordt gebruikt voor het programmeren, debuggen en beheren van code.

Voor de eindgebruiker betekent dit dat alleen een apparaat met browser en een internetverbinding nodig zijn. Alle complexe berekeningen, dataverwerking en serverbeheer worden volledig door Vercel afgehandeld. Dit maakt de applicatie toegankelijk en consistent in prestaties, ongeacht de lokale specificaties van de hardware.

7. 2 Indicatieve kosten bij klein gebruik

Tabel 1: Kosteninschatting

Component	Free tier/ gratis limiet	Indicatieve kosten per maand (afhankelijk van gebruik, gratis tot starter plan)
Vercel (hosting en frontend)	Draaien van de website is gratis tot 100 GB	\$0- \$20
Neon Database	Opslaan van gebruikersinformatie is gratis tot 0.5 GB opslag	\$0- \$15
GROQ	Gratis tot: 30 requests per minuut 1000 requests per dag 12000 input/ 6000 output tokens per minuut	\$0- \$50
	Totaal	Circa \$ 85